

Deep Learning (CAI-461)

Objectives:

- Action-advantage function
- Optimality
- Policy evaluation

Action-advantage function:

Another type of value function is derived from the previous two. The *action-advantage function*, also known as **advantage function, A-function, or $A\pi(s, a)$** , is the difference between the action-value function of action a in state s and the state-value function of state s under policy π .

The advantage function describes how much better it is to take action a instead of following policy π : the advantage of choosing action a over the default action. Look at the different value functions for a (dumb) policy in the SWF environment. Remember, these values depend on the policy. In other words, the $Q\pi(s, a)$ assumes you'll follow policy π (always left in the following example) and right after taking action a in state s .

(1) The advantage of action a in state s under policy π ...

$$a_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$$

(2) ... is the difference between the value of that action and the value of the state s , both under policy π .

State-value, action-value, and action-advantage functions

(1) Notice how $q_{\pi}(s, a)$ allows us to improve policy π , by showing the highest valued action under the policy.

(2) Also notice there's no advantage for taking the same action as policy π recommends.

H	←	←	START	←	←	G	← π
0	1	2	3	4	5	6	

0.0	0.002	0.011	0.036	0.11	0.332	0.0	← $V_{\pi}(s)$
0	1	2	3	4	5	6	

0	0	0	0	0	0	0	←
0.0	0.002	0.011	0.036	0.11	0.332	0.0	← $Q_{\pi}(s, a)$
1	0	1	0	1	0	1	→
0	1	2	3	4	5	6	

0	0	0	0	0	0	0	←
0.0	0.0	0.0	0.0	0.0	0.0	0.0	← $A_{\pi}(s, a)$
1	0	1	0	1	0	1	→
0	1	2	3	4	5	6	

Optimality

Policies, state-value functions, action-value functions, and action-advantage functions are the components we use to describe, evaluate, and improve behaviors. We call it *optimality* when these components are the best they can be.

An **optimal policy** is a policy that for every state can obtain expected returns greater than or equal to any other policy. An **optimal state-value function** is a state-value function with the maximum value across all policies for all states. Likewise, an **optimal action-value function** is an action-value function with the maximum value across all policies for all state-action pairs. The **optimal action-advantage function** follows a similar pattern, but notice an optimal advantage function would be equal to or less than zero for all state-action pairs, since no action could have any advantage from the optimal state-value function.

Also, notice that although there could be more than one optimal policy for a given MDP, there can only be one optimal state-value function, optimal action-value function, and optimal action-advantage function.

You may also notice that if you had the optimal V-function, you could use the MDP to do a one-step search for the optimal Q-function and then use this to build the optimal policy. On the other hand, if you had the optimal Q-function, you don't need the MDP at all. You could use the optimal Q-function to find the optimal V-function by just taking the maximum over the actions. And you could obtain the optimal policy using the optimal Q-function by taking the argmax over the actions.

The Bellman Optimality equation

(1) The optimal state-value function ... $v_*(s) = \max_{\pi} v_{\pi}(s), \forall s \in S$ (a) ... is the state-value function with the highest value across all policies.

(3) Likewise, the optimal action-value function is the action-value function with the highest values. $q_*(s, a) = \max_{\pi} q_{\pi}(s, a), \forall s \in S, \forall a \in A(s)$

(4) The optimal state-value function can be obtained this way. $v_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')]$

(5) We take the max action ... (6) ... of the weighted sum of the reward and discounted optimal value of the next state.

(7) Similarly, the optimal action-value function can be obtained this way. $q_*(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]$

(8) Notice how the max is now on the inside.

Policy evaluation

We talked about comparing policies in the previous section. We established that policy π is better than or equal to policy π' if the expected return is better than or equal to π' for all states. Before we can use this definition, however, we must devise an algorithm for evaluating an arbitrary policy. Such an algorithm is known as an *iterative policy evaluation* or just *policy evaluation*.

The policy-evaluation algorithm consists of calculating the V-function for a given policy by sweeping through the state space and iteratively improving estimates. We refer to the type of algorithm that takes in a policy and outputs a value function as an algorithm that solves the **prediction problem**, which is calculating the values of a predetermined policy.

(1) The policy-evaluation algorithm consist of the iterative approximation of the state-value function of the policy under evaluation. The algorithm converges as k approaches infinity.

(2) Initialize $v_0(s)$ for all s in S arbitrarily, and to 0 if s is terminal. Then, increase k and iteratively improve the estimates by following the equation below.

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_k(s') \right]$$

(3) Calculate the value of a state s as the weighted sum of the reward and the discounted estimated value of the next state s' .

Using this equation, we can iteratively approximate the true V-function of an arbitrary policy. The iterative policy-evaluation algorithm is guaranteed to converge to the value function of the policy if given enough iterations, more concretely as we approach infinity. In practice, however, we use a small threshold to check for changes in the value function we're approximating. Once the changes in the value function are less than this threshold, we stop.

Let's see how this algorithm works in the SWF environment, for the always-left policy.

Initial calculations of policy evaluation

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_k(s') \right]$$

(1) We have a deterministic policy, so this part is 1. $\rightarrow \pi$

(2) Let's use gamma of 1. $\rightarrow \gamma$

(3) An Always-left policy. $\rightarrow \pi$

H ₀	← ₁	← ₂	← ₃ START	← ₄	← ₅	G ₆
-----------------------	----------------	----------------	-----------------------------	----------------	----------------	-----------------------

State 5, Iteration 1 (initialized to 0 in iteration 0):

$$v_1^\pi(5) = p(s'=4 | s=5, a=\text{Left}) * [R(5, \text{Left}, 4) + v_0^\pi(4)] +$$

$$p(s'=5 | s=5, a=\text{Left}) * [R(5, \text{Left}, 5) + v_0^\pi(5)] +$$

$$p(s'=6 | s=5, a=\text{Left}) * [R(5, \text{Left}, 6) + v_0^\pi(6)]$$

$$v_1^\pi(5) = 0.50 * (0+0) + 0.33 * (0+0) + 0.166 * (1+0) = 0.166$$

(4) Yep, this is the value of state 5 after 1 iteration of policy evaluation ($v_1^\pi(5)$).

You then calculate the values for all states 0–6, and when done, move to the next iteration.

Notice that to calculate $V2^\pi(s)$ you'd have to use the estimates obtained in the previous iteration, $V1^\pi(s)$. This technique of calculating an estimate from an estimate is referred to as **bootstrapping**, and it's a widely used technique in RL (including DRL).

Also, it's important to notice that the k 's here are iterations across estimates, but they're not interactions with the environment. These aren't episodes that the agent is out and about selecting actions and observing the environment. These aren't time steps either. Instead, these are the iterations of the iterative policy-evaluation algorithm. Do a couple more of these estimates.

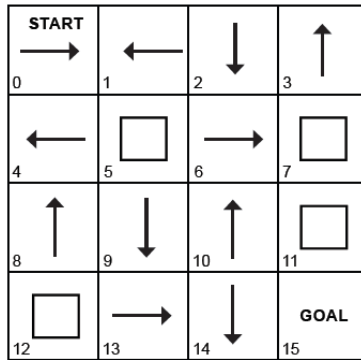
The following table shows you the results you should get.

k	$V^{\pi}(0)$	$V^{\pi}(1)$	$V^{\pi}(2)$	$V^{\pi}(3)$	$V^{\pi}(4)$	$V^{\pi}(5)$	$V^{\pi}(6)$
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0.1667	0
2	0	0	0	0	0.0278	0.2222	0
3	0	0	0	0.0046	0.0463	0.2546	0
4	0	0	0.0008	0.0093	0.0602	0.2747	0
5	0	0.0001	0.0018	0.0135	0.0705	0.2883	0
6	0	0.0003	0.0029	0.0171	0.0783	0.2980	0
7	0	0.0006	0.0040	0.0202	0.0843	0.3052	0
8	0	0.0009	0.0050	0.0228	0.0891	0.3106	0
9	0	0.0011	0.0059	0.0249	0.0929	0.3147	0
10	0	0.0014	0.0067	0.0267	0.0959	0.318	0
...	...	...	...	...	...	...	...
104	0	0.0027	0.011	0.0357	0.1099	0.3324	0

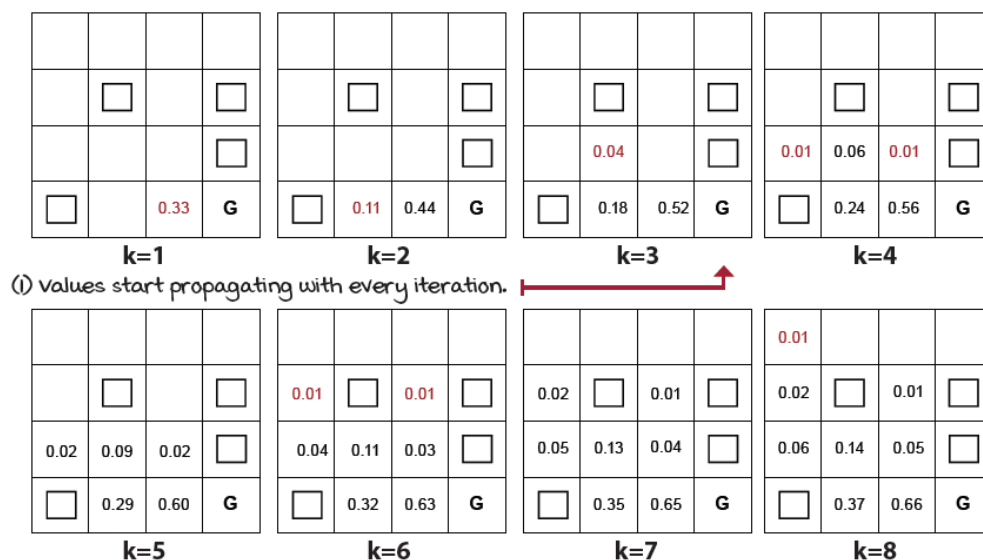
What are several of the things the resulting state-value function tells us? Well, to begin with, we can say we get a return of 0.0357 in expectation when starting an episode in this environment and following the always-left policy. Pretty low. We can also say, that even when we find ourselves in state 1 (the leftmost non-terminal state), we still have a chance, albeit less than one percent, to end up in the GOAL cell (state 6). To be exact, we have a 0.27% chance of ending up in the GOAL state when we're in state 1. And we select left all the time! Pretty interesting.

Interestingly also, due to the stochasticity of this environment, we have a 3.57% chance of reaching the GOAL cell (remember this environment has 50% action success, 33.33% no effects, and 16.66% backward). Again, this is when under an always-left policy. Still, the Left action could send us right, then right and right again, or left, right, right, right, right, and so on. Think about how the probabilities of trajectories combine. Also, pay attention to the iterations and how the values propagate backward from the reward (transition from state 5 to state 6) one step at a time. This backward propagation of the values is a common characteristic among RL algorithms and comes up again several times.

Let's now run policy evaluation in the randomly generated policy presented earlier for the FL environment.



Policy evaluation on the randomly generated policy for the FL environment



State-value function of the randomly generated policy

START →	←	↓	↑
0.0955	0.0471	0.0470	0.0456
←	□	→	□
0.1469		0.0498	
↑	↓	↑	□
0.2028	0.2647	0.1038	
□	→	↓	GOAL
	0.4957	0.7417	

(i) After 218 interactions, policy evaluation converges to these values (using a $1e-10$ minimum change in values as a stopping condition).

This final state-value function is the state-value function for this policy. Note that even though this is still an estimate, because we're in a discrete state and action spaces, we can assume this to be the actual value function when using gamma of 0.99.

Results of Policy evaluation are given below

Results of policy evolution

The Go-get-it policy:

START →	→	↓	←
0.0342	0.0231	0.0468	0.0231
↓	□	↓	□
0.0463		0.0957	
→	→	↓	□
0.0940	0.2386	0.2901	
□	→	→	GOAL
	0.4329	0.6404	

(1) The state-value function of this policy converges after 66 iterations. The policy reaches the goal state a mere 3.4% of the time.

The Careful policy:

START ←	↑	↑	↑
0.4079	0.3754	0.3543	0.3438
←	□	↑	□
0.4263		0.1169	
↑	↓	←	□
0.4454	0.4840	0.4328	
□	→	→	GOAL
	0.5884	0.7107	

(2) For this policy, the state-value function converges after 546 iterations. The policy reaches the goal 53.70% of the time!

(3) By the way, I calculated these values empirically by running the policies 100 times. Therefore, these values are noisy, but you get the idea.